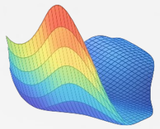


Intelligent Systems,
MIPT

Natural Language Processing

L8 / 07.04.26

- Course page
- GitHub

Content

Lecture 8: 5D Parallelism, Mixture of Experts

- Tensor & Context & Pipeline Parallelism
- 5D Parallelism
- Mixture of Experts (MoE)

Recap: last lecture

- Single-GPU: weights + grads + optimizer + activations
- Mixed Precision: BF16 compute, FP32 master weights for stability
- Data Parallelism: replicate model, parallelize micro-batches, all-reduce gradients
- ZeRO-1/2/3 (FSDP): shard optimizer states/gradients/parameters across DP ranks

Recap: last lecture

- Single-GPU: weights + grads + optimizer + activations
- Mixed Precision: BF16 compute, FP32 master weights for stability
- Data Parallelism: replicate model, parallelize micro-batches, all-reduce gradients
- ZeRO-1/2/3 (FSDP): shard optimizer states/gradients/parameters across DP ranks

What remained unsolved:

- ZeRO doesn't shard activations — they still blow up with long sequences
- DP communication overhead grows at 500+ GPUs
- A single layer might not fit on one GPU for very large models

Tensor Parallelism: intro

- We could shard weights, gradients, and optimizers states as well as activations and without the need to gather them all prior to the computation

Tensor Parallelism: intro

- **We could shard weights, gradients, and optimizers states as well as activations and without the need to gather them all prior to the computation**
- **Let's remember:**

1. $A \cdot B = A \cdot \begin{bmatrix} B_1 & B_2 & \cdots \end{bmatrix} = \begin{bmatrix} AB_1 & AB_2 & \cdots \end{bmatrix}$
2. $A \cdot B = \begin{bmatrix} A_1 & A_2 & \cdots \end{bmatrix} \begin{bmatrix} B_1 \\ B_2 \\ \vdots \end{bmatrix} = \sum_{i=1}^{n} A_i B_i$

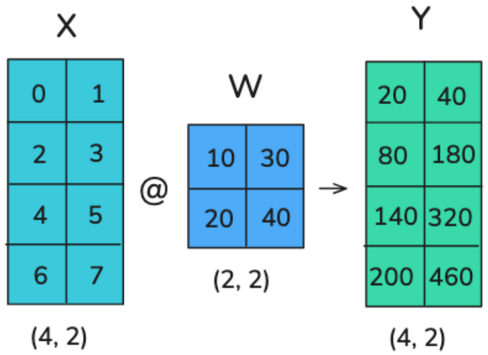
Tensor Parallelism: intro

- We could shard weights, gradients, and optimizers states as well as activations and without the need to gather them all prior to the computation
- Let's remember:

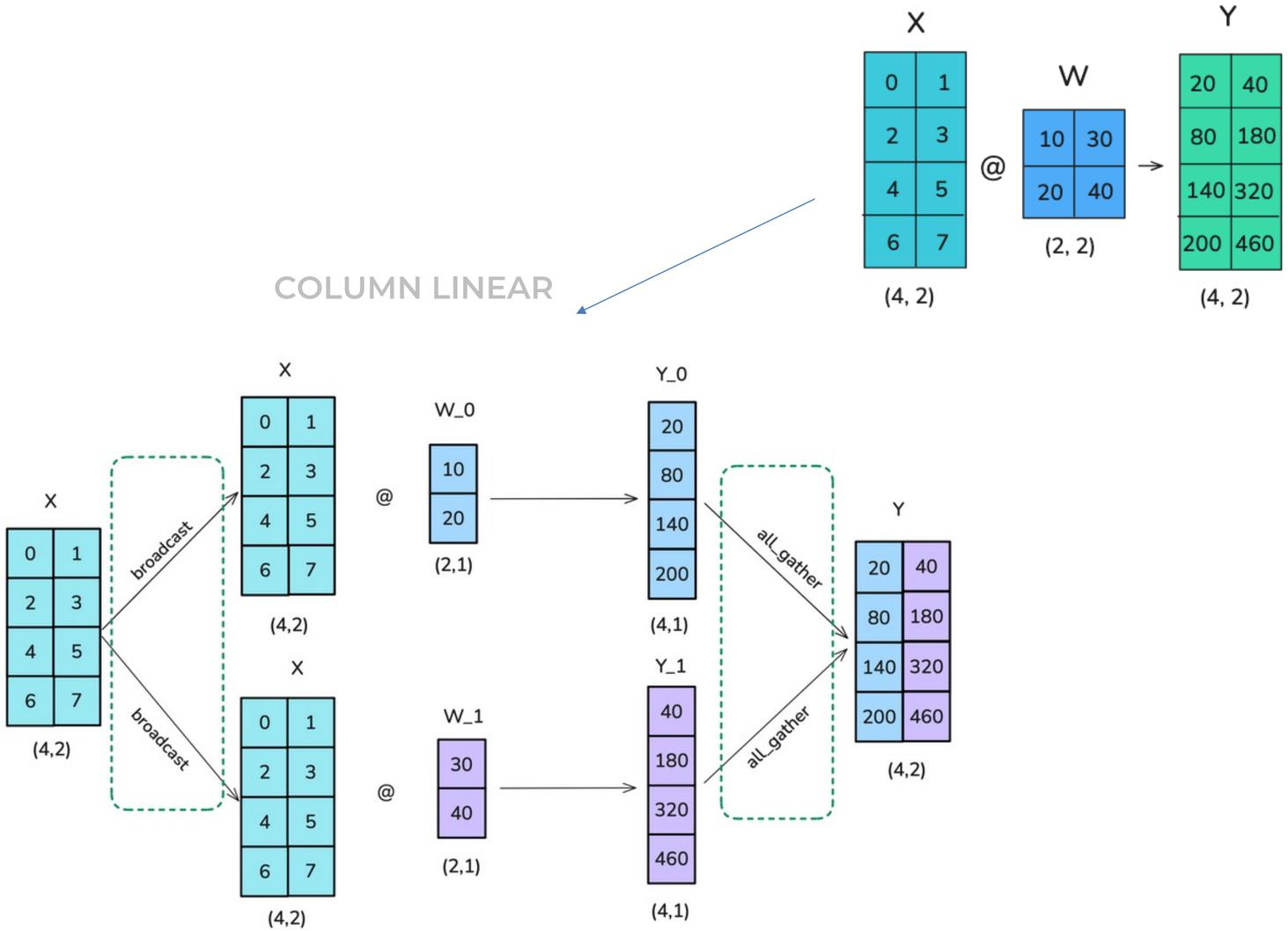
1. $A \cdot B = A \cdot \begin{bmatrix} B_1 & B_2 & \cdots \end{bmatrix} = \begin{bmatrix} AB_1 & AB_2 & \cdots \end{bmatrix}$
2. $A \cdot B = \begin{bmatrix} A_1 & A_2 & \cdots \end{bmatrix} \begin{bmatrix} B_1 \\ B_2 \\ \vdots \end{bmatrix} = \sum_{i=1}^{n} A_i B_i$

- **Idea:** we can compute matrix product by either 1) multiplying each column of B individually or 2) multiplying each row individually and combining the results

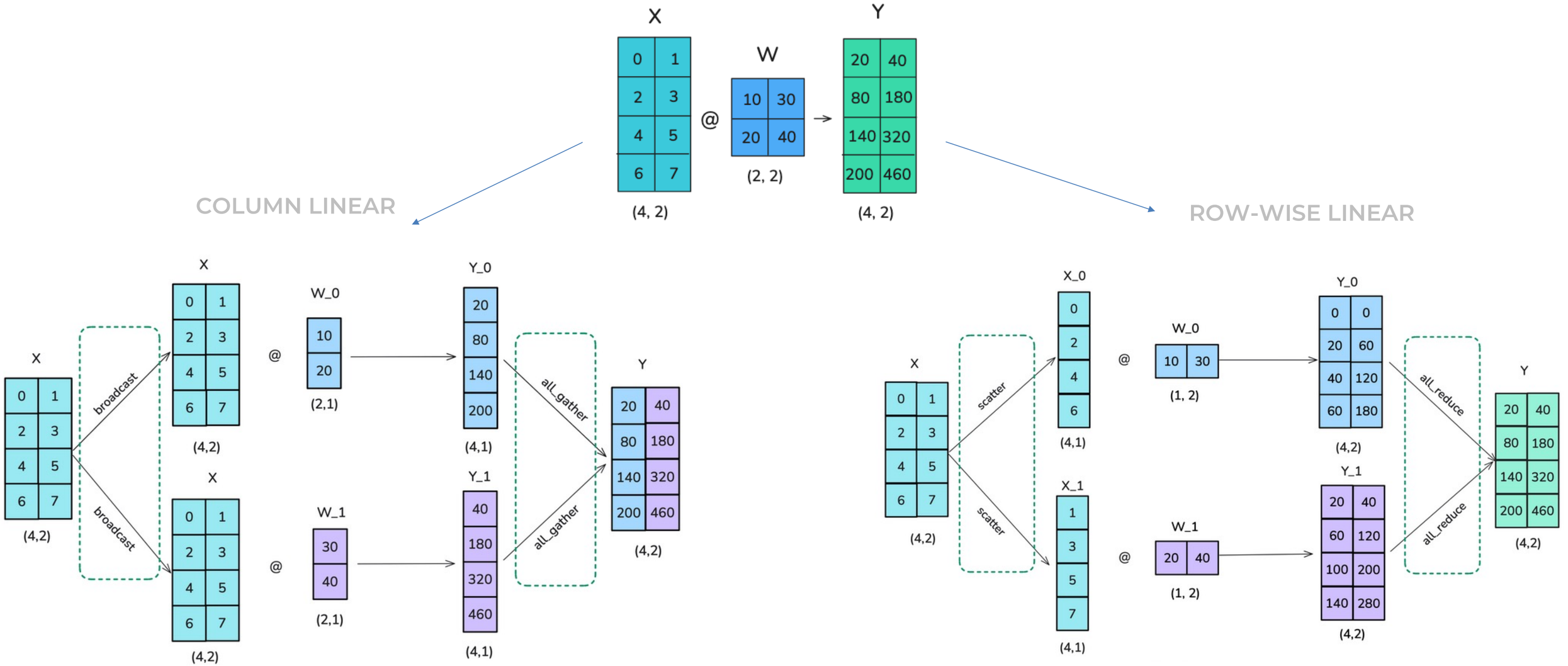
Operations example



Operations example

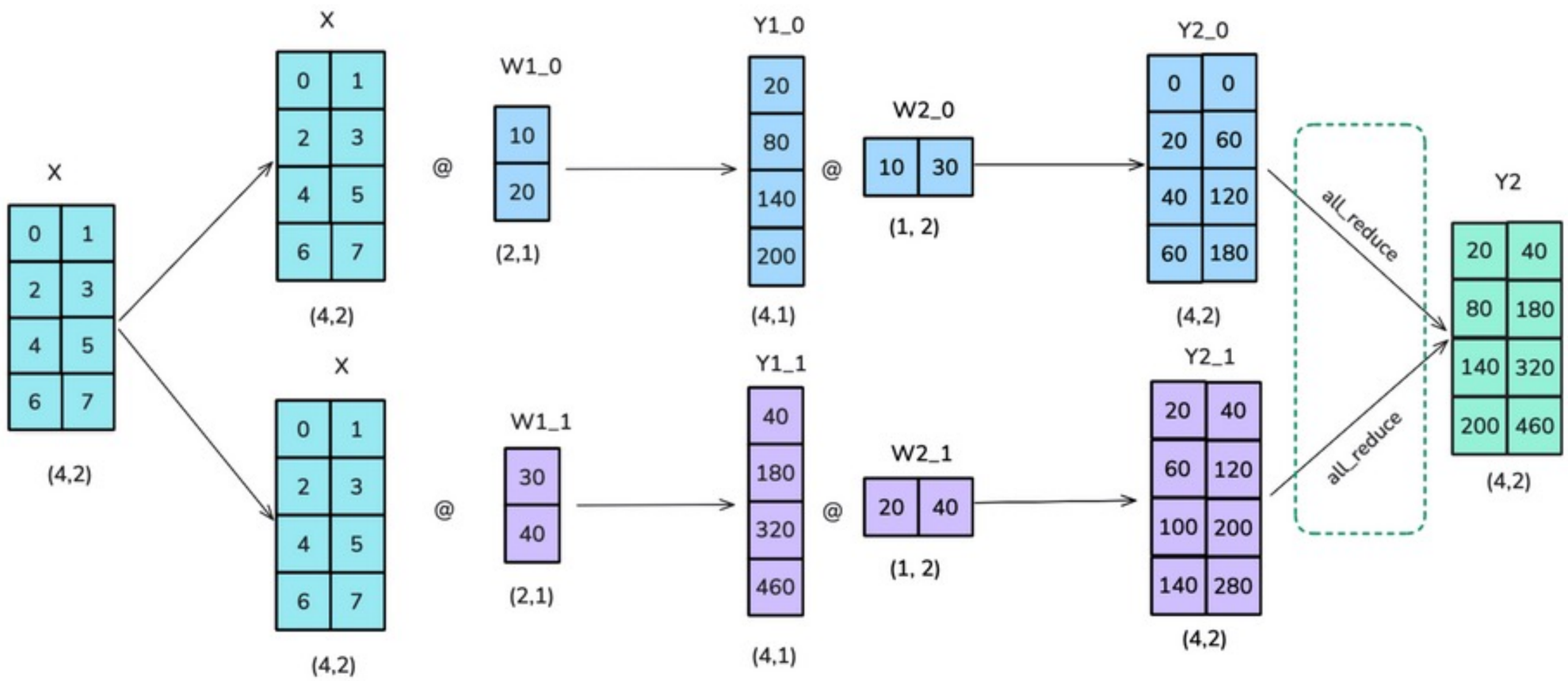


Operations example



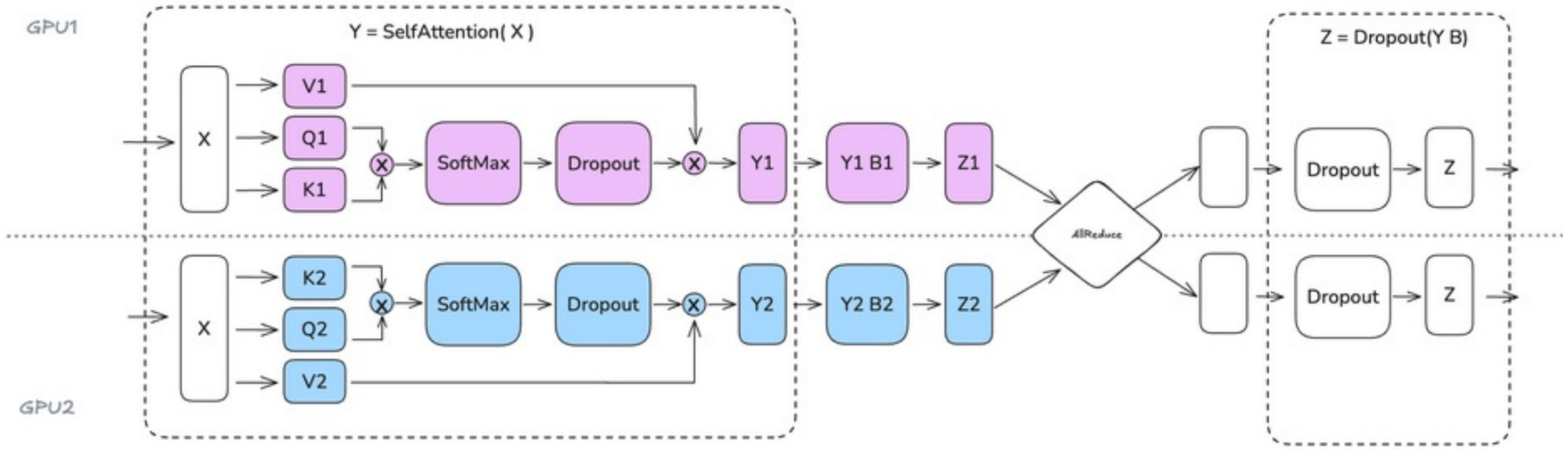
Tensor Parallelism in a Transformer Block

- **FFN:**



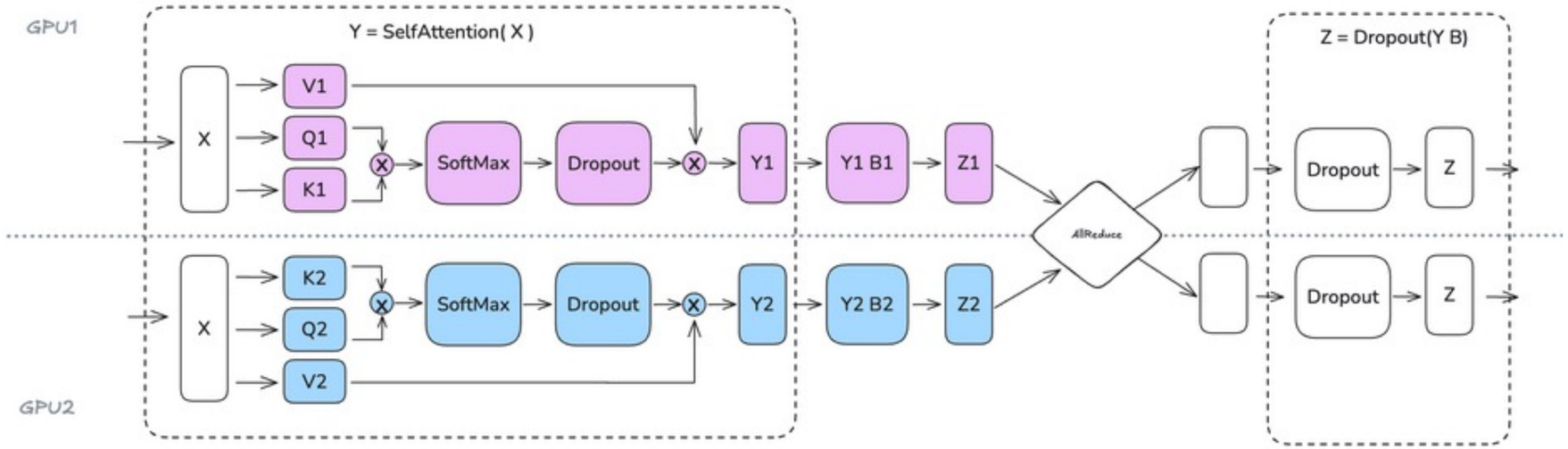
Tensor Parallelism in a Transformer Block

- **Multi-Head Attention:**

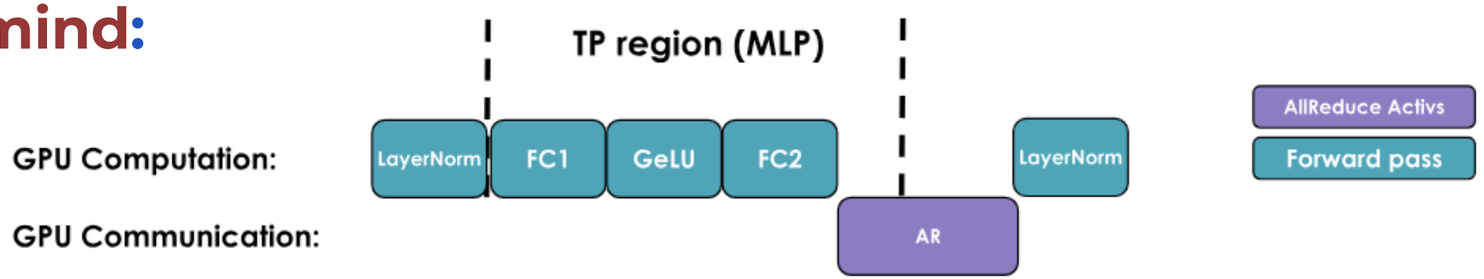


Tensor Parallelism in a Transformer Block

- **Multi-Head Attention:**



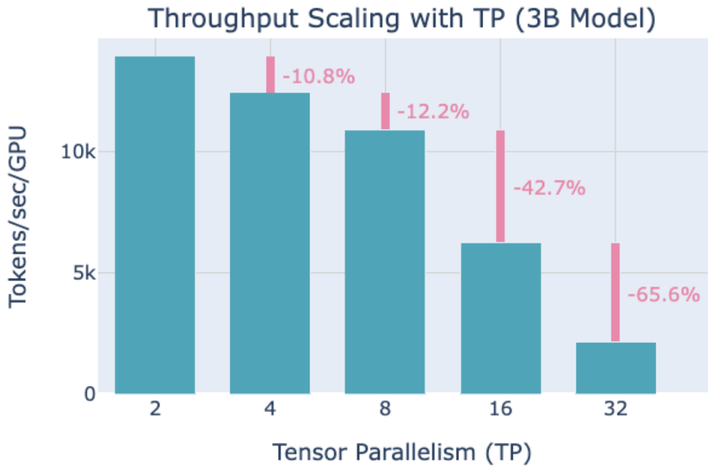
- **Keep in mind:**

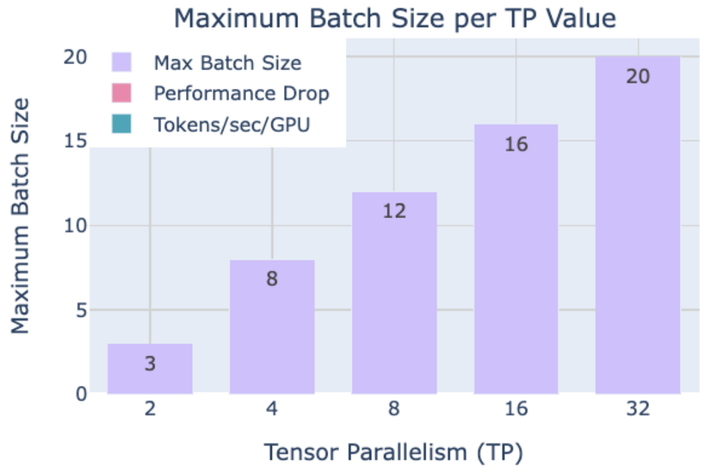


Tensor Parallelism Trade-off

- **TP introduces significant communication requirements that heavily depend on the network infrastructure**

In practice: TP ≤ 8 (within one node, NVLink 900 GB/s). Beyond — steep degradation

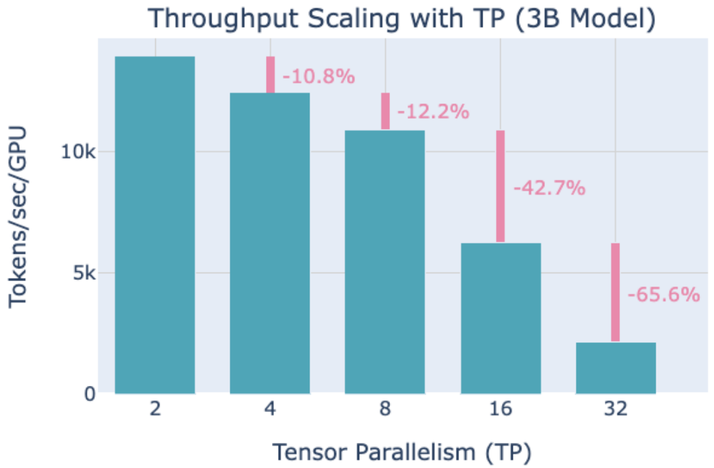


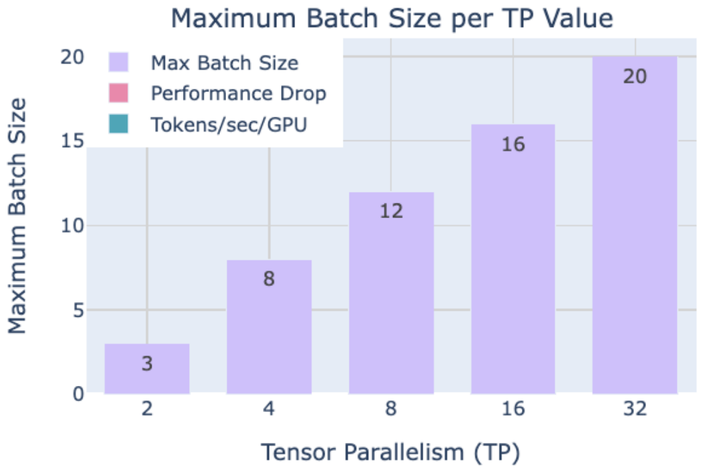


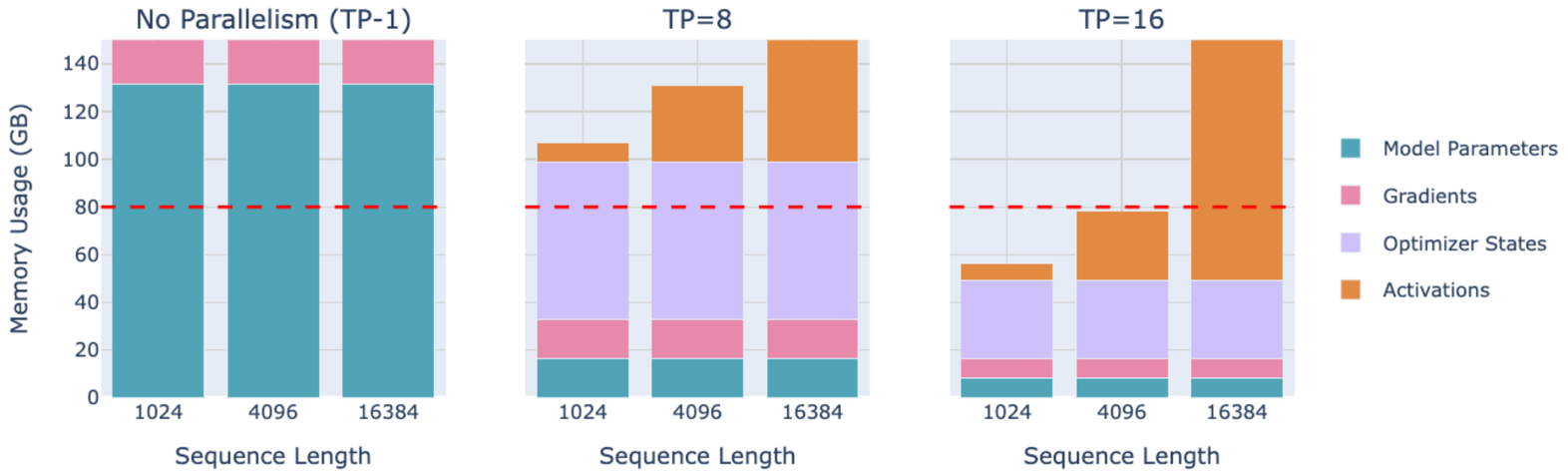
Tensor Parallelism Trade-off

- **TP introduces significant communication requirements that heavily depend on the network infrastructure**

In practice: TP ≤ 8 (within one node, NVLink 900 GB/s). Beyond — steep degradation







- **Memory usage for 70B**

Context Parallelism

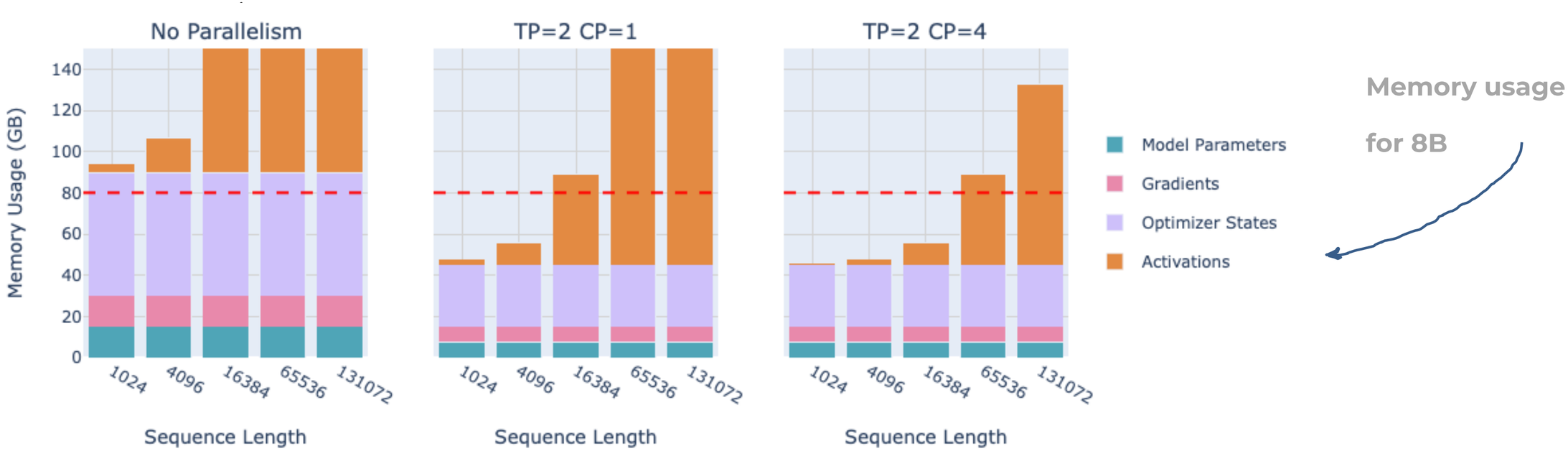
- **Problem:** Attention memory is $O(seq^2)$ — at 128K tokens this dominates everything. Even if we use full recomputation of the activations, we still need to hold in memory some activations at the layer boundaries which scale linearly with sequence length

Context Parallelism

- **Problem:** Attention memory is $O(seq^2)$ — at 128K tokens this dominates everything. Even if we use full recomputation of the activations, we still need to hold in memory some activations at the layer boundaries which scale linearly with sequence length
- **Idea:** split the sequence into chunks, each GPU holds one chunk

Context Parallelism

- **Problem:** Attention memory is $O(seq^2)$ — at 128K tokens this dominates everything. Even if we use full recomputation of the activations, we still need to hold in memory some activations at the layer boundaries which scale linearly with sequence length
- **Idea:** split the sequence into chunks, each GPU holds one chunk



Context Parallelism: communication

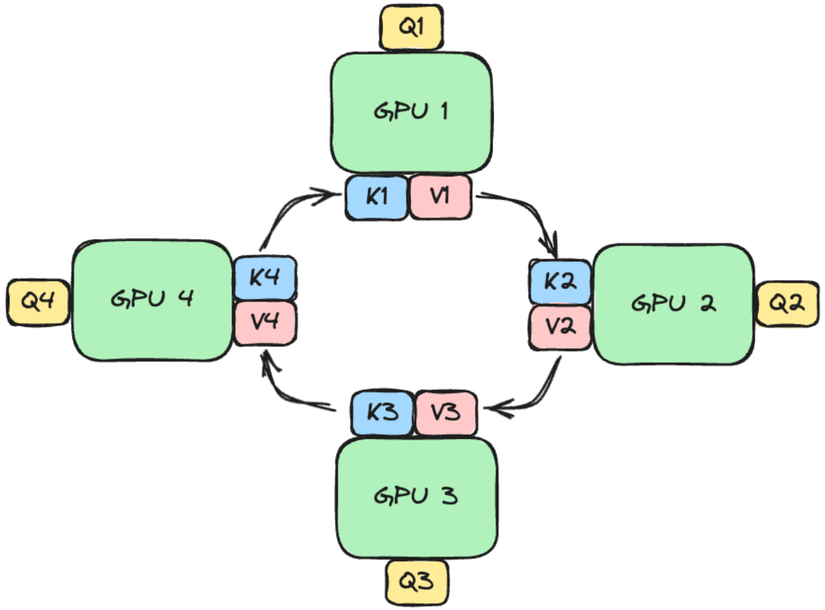
- Splitting the sequence doesn't affect most modules like MLP and LayerNorm, where each token is processed independently. What about Attention?

Context Parallelism: communication

- Splitting the sequence doesn't affect most modules like MLP and LayerNorm, where each token is processed independently. What about Attention?
- CP splits the inputs along the sequence dimension across GPUs, the attention module will require full communication between GPUs to exchange the necessary key/value data

Context Parallelism: communication

- Splitting the sequence doesn't affect most modules like MLP and LayerNorm, where each token is processed independently. What about Attention?
- CP splits the inputs along the sequence dimension across GPUs, the attention module will require full communication between GPUs to exchange the necessary key/value data
- Solution: Ring Attention



Pipeline Parallelism

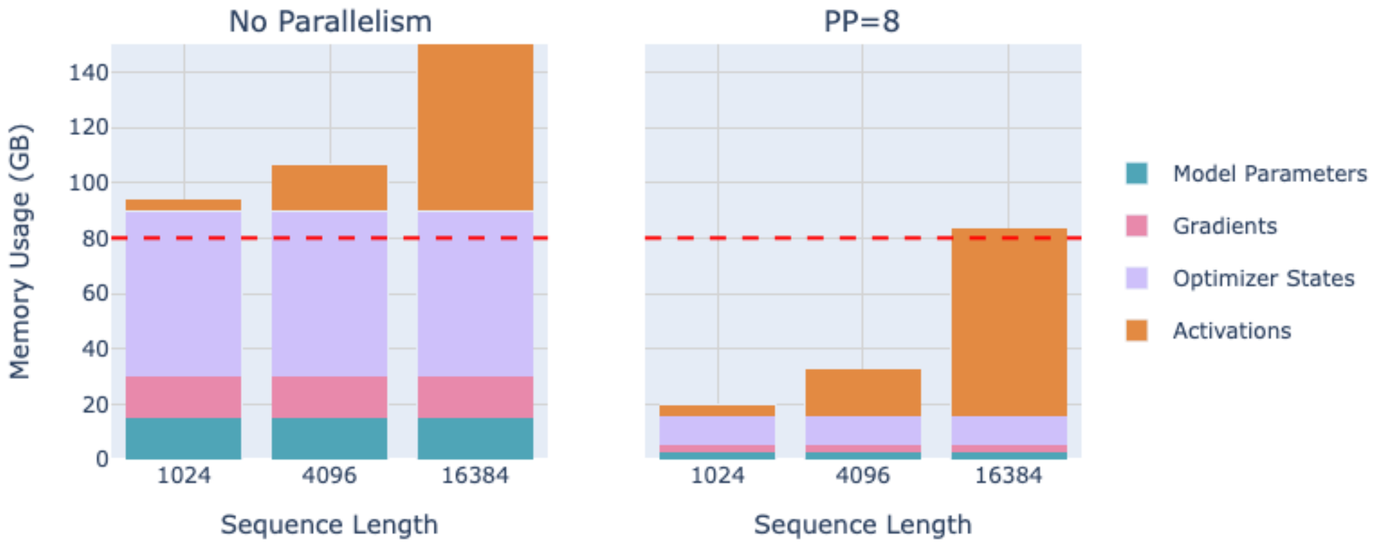
- **Problem:** In the TP section we saw that trying to scale TP past the number of GPUs per single node (4 or 8) hit a lower bandwidth network which can quite strongly impair performances

Pipeline Parallelism

- **Problem:** In the TP section we saw that trying to scale TP past the number of GPUs per single node (4 or 8) hit a lower bandwidth network which can quite strongly impair performances
- **Idea:** split our model's layers across multiple GPUs

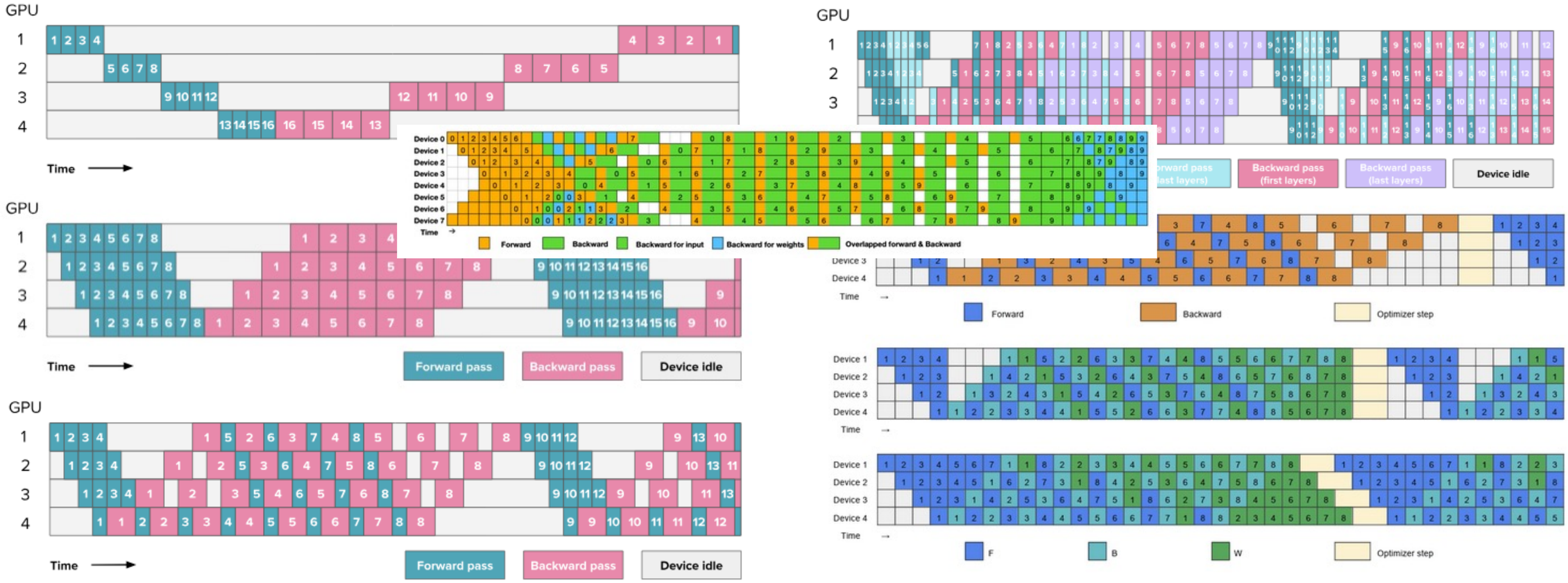
Pipeline Parallelism

- **Problem:** In the TP section we saw that trying to scale TP past the number of GPUs per single node (4 or 8) hit a lower bandwidth network which can quite strongly impair performances
- **Idea:** split our model's layers across multiple GPUs
- Each GPU still needs to process the full batch of data. The activations from one GPU's layers will be sent to the next GPU to continue the forward pass.



Pipeline Parallelism: communication

- **A new type of communication pattern: instead of communicating parameters like we did with ZeRO-3 in DP, we're now passing activation tensors sequentially between GPUs in a "pipeline"**



5D Parallelism

- Data Parallelism (DP) – along the batch dimension
- Tensor Parallelism (TP) - along the hidden dimension
- Sequence and Context Parallelism (SP/CP) - along the sequence dimension
- Pipeline Parallelism (PP) - along the model layers
- Expert Parallelism (EP) - along the model experts

5D Parallelism

- Data Parallelism (DP) – along the batch dimension
- Tensor Parallelism (TP) - along the hidden dimension
- Sequence and Context Parallelism (SP/CP) - along the sequence dimension
- Pipeline Parallelism (PP) - along the model layers
- Expert Parallelism (EP) - along the model experts

Technique	When	What it saves	Cost
Tensor Parallel	Layer > GPU mem	Activations + params	All-reduce in path
Sequence Parallel	Always with TP	Act. mem (LN/Drop)	Free vs TP
Pipeline Parallel	Model > 1 node	Layers across GPUs	Bubble overhead
Context Parallel	Seq > 8–16K	Attention memory	Ring send/recv
Data Parallel	Multiple GPUs	Throughput ×N	All-reduce comm
ZeRO-1/2/3	Model > 1 GPU	Params/grads/opt	Extra comm

Mixture of Experts: motivation

- **Reminder:** in Transformer, almost half of the parameters reside within the feed- forward neural network layers

Mixture of Experts: motivation

- **Reminder: in Transformer, almost half of the parameters reside within the feed- forward neural network layers**

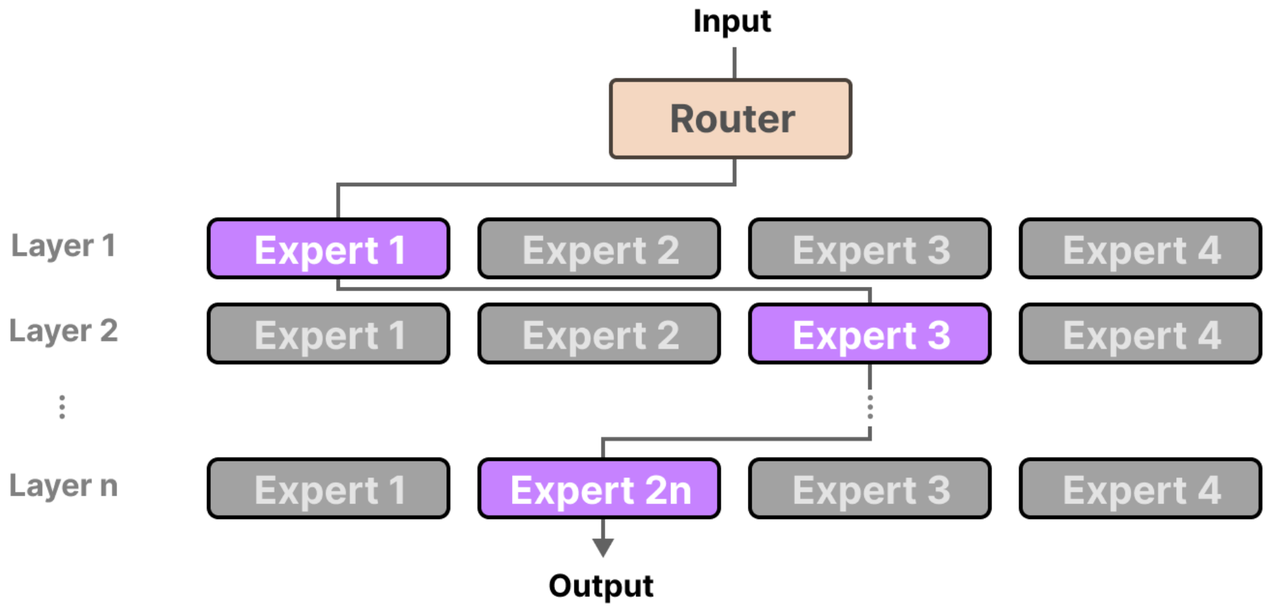
Doubling parameters → doubling FLOPs for every forward pass. Training a 1T dense model is practically infeasible.

All parameters must be on accelerators. Dense trillion-param models exceed GPU/TPU memory.

The same neurons handle all token types: syntax, semantics, math, code. Suboptimal for heterogeneous text.

Mixture of Experts

- **Idea:** replace the FFN sublayer of a Transformer with N parallel FFN experts. Each expert is just a feed-forward network. A gating network selects only k experts per token (sparse activation)



What each expert learns?

- **How we define an expert?** Usually each expert is responsible for dealing with some syntactic or semantic structures, especially encoder experts

What each expert learns?

- **How we define an expert?** Usually each expert is responsible for dealing with some syntactic or semantic structures, especially encoder experts

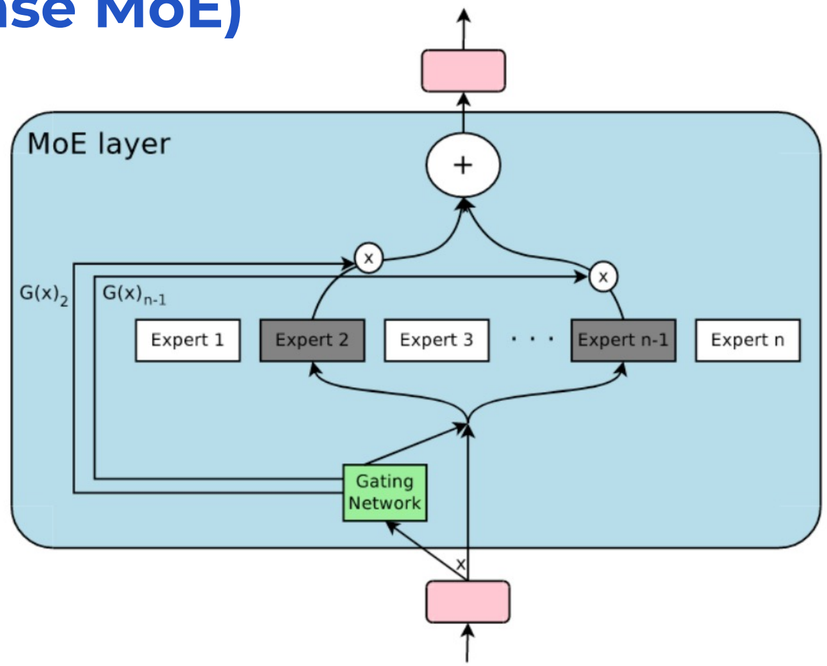
- We might expect some monolinguality of experts for multilingual dataset, but it turns out the experts operate on less abstract level and there is no single expert specialized in any given language

Expert specialization	Expert position	Routed tokens
Sentinel tokens	Layer 1	been <extra_id_4><extra_id_7>floral to <extra_id_10><extra_id_12><extra_id_15> <extra_id_17><extra_id_18><extra_id_19>...
	Layer 4	<extra_id_0><extra_id_1><extra_id_2> <extra_id_4><extra_id_6><extra_id_7> <extra_id_12><extra_id_13><extra_id_14>...
	Layer 6	<extra_id_0><extra_id_4><extra_id_5> <extra_id_6><extra_id_7><extra_id_14> <extra_id_16><extra_id_17><extra_id_18>...
Punctuation	Layer 2	, , , , , , , , - , , , ,).)
	Layer 6	, , , , : . : , & , & & ? & - , , ? , , , . <extra_id_27>
Conjunctions and articles	Layer 3	The the the the the the the the the The the the the the the The the the the
	Layer 6	a and and and and and and and or and a and . the the if ? a designed does been is not
Verbs	Layer 1	died falling identified fell closed left posted lost felt left said read miss place struggling falling signed died falling designed based disagree submitted develop
Visual descriptions *color, spatial position*	Layer 0	her over her know dark upper dark outer center upper blue inner yellow raw mama bright bright over open your dark blue
Proper names	Layer 1	A Mart Gr Mart Kent Med Cor Tri Ca Mart R Mart Lorraine Colin Ken Sam Ken Gr Angel A Dou Now Ga GT Q Ga C Ko C Ko Ga G
Counting and numbers *written and numerical forms*	Layer 1	after 37 19. 6. 27 I I Seven 25 4, 54 I two dead we Some 2012 who we few lower each

Defining Gating

- **If we want to take into account all voices (dense MoE)**

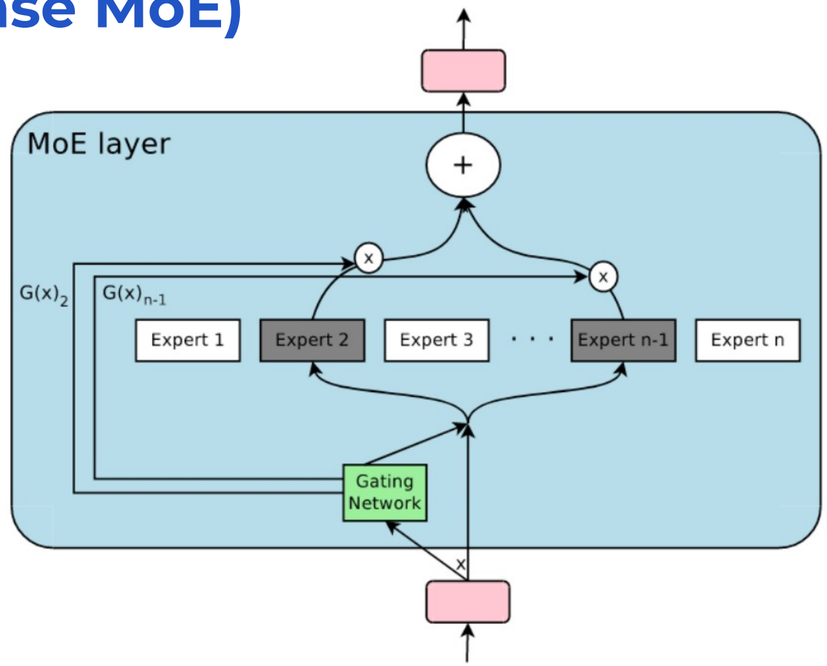
$$G(\mathbf{x}) = \text{softmax}(\mathbf{x} \cdot \mathbf{W}_g)$$



Defining Gating

- **If we want to take into account all voices (dense MoE)**

$$G(\mathbf{x}) = \text{softmax}(\mathbf{x} \cdot \mathbf{W}_g)$$



- **Sparse MoE: let's take into account voices of k experts**

1. We add some noise

$$H(x)_i = (x \cdot W_{\text{g}})_i + \text{StandardNormal}() \cdot \text{Softplus}((x \cdot W_{\text{noise}})_i)$$

2. We only pick the top k

$$\text{KeepTopK}(v, k)_i = \begin{cases} v_i & \text{if } v_i \text{ is in the top } k \text{ elements of } v, \\ -\infty & \text{otherwise.} \end{cases}$$

3. We apply the softmax.

$$G(x) = \text{Softmax}(\text{KeepTopK}(H(x), k))$$

DeepSeek MoE

- **DeepSeek Team introduced two types of experts, shared experts (first sum) and routed experts (second sum). Weights for routed experts defined by normalized relevance, where relevance $g_{i,t}$:**

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \cdot \mathrm{FFN}_i^{(r)}(\mathbf{u}_t)$$

DeepSeek MoE

- DeepSeek Team introduced two types of experts, shared experts (first sum) and routed experts (second sum). Weights for routed experts defined by normalized relevance, where relevance $g_{i,t}$:

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \cdot \mathrm{FFN}_i^{(r)}(\mathbf{u}_t)$$

- Each expert is defined by centroid, and to calculate relevance of input with this expert we use dot product with the input:

$$g'_{i,t} = \begin{cases} s_{i,t} & \text{if } s_{i,t} \in \mathrm{Top}_k(\{s_{j,t} | 1 \leq j \leq N_t\}, k_r) \\ 0 & \text{otherwise} \end{cases}$$

DeepSeek MoE

- DeepSeek Team introduced two types of experts, shared experts (first sum) and routed experts (second sum). Weights for routed experts defined by normalized relevance, where relevance $g_{i,t}$:

$$\mathbf{h}'_t = \mathbf{u}_t + \sum_{i=1}^{N_s} \mathrm{FFN}_i^{(s)}(\mathbf{u}_t) + \sum_{i=1}^{N_r} g_{i,t} \cdot \mathrm{FFN}_i^{(r)}(\mathbf{u}_t)$$

- Each expert is defined by centroid, and to calculate relevance of input with this expert we use dot product with the input:

$$g'_{i,t} = \begin{cases} s_{i,t} & \text{if } s_{i,t} \in \mathrm{Top}_k(\{s_{j,t} | 1 \leq j \leq N_t\}, k_r) \\ 0 & \text{otherwise} \end{cases}$$

- There might be a **problem** that some experts might be more relevant to inputs and they will be overloaded...

Load Balancing

- **Routing alone breaks training. Without regularization, all tokens route to the same 1-2 experts. This is called expert collapse:**

  > Popular experts get huge gradients → get better → get even more tokens

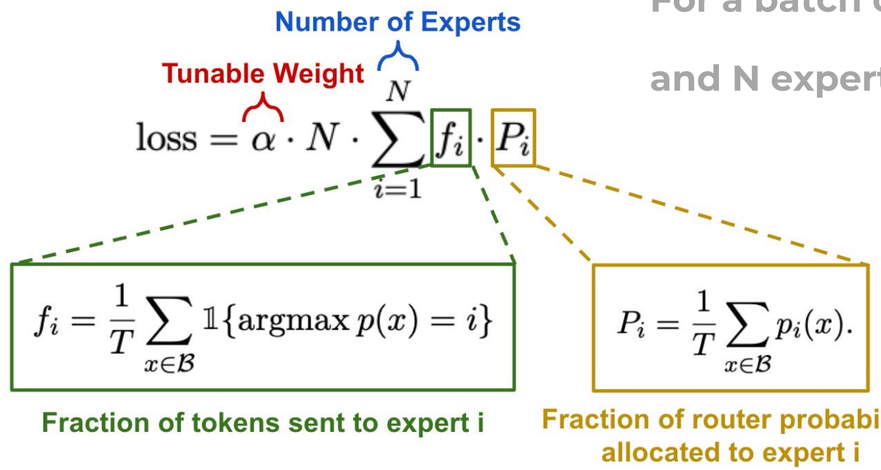
  > Unpopular experts starve → become useless → effectively wasted parameters

  > Model finally degenerates into a dense model with wasted capacity

Load Balancing

- **Routing alone breaks training. Without regularization, all tokens route to the same 1-2 experts. This is called expert collapse:**
  - > Popular experts get huge gradients → get better → get even more tokens
  - > Unpopular experts starve → become useless → effectively wasted parameters
  - > Model finally degenerates into a dense model with wasted capacity
- **Imbalance is first found in the choice of experts but also in the distributions of tokens that are sent to the expert. Suggested to use Auxiliary Load Balancing Loss**

For a batch of T tokens and N experts, define:



Load Balancing

- **Routing alone breaks training. Without regularization, all tokens route to the same 1-2 experts. This is called expert collapse:**

  > Popular experts get huge gradients → get better → get even more tokens

  > Unpopular experts starve → become useless → effectively wasted parameters

  > Model finally degenerates into a dense model with wasted capacity

- **Imbalance is first found in the choice of experts but also in the distributions of tokens that are sent to the expert. Suggested to use Auxiliary Load Balancing Loss**

  > The factor **N** normalizes so that perfect balances gives tunable weight regardless of **N**. The product in sum minimizes when all f_i are uniform. Since f_i is not differentiable, gradient flows only through P_i, pushing router to spread prob. mass more evenly

For a batch of T tokens and N experts, define:

Number of Experts

Tunable Weight

$$\text{loss} = \alpha \cdot N \cdot \sum_{i=1}^{N} f_i \cdot P_i$$

$$f_i = \frac{1}{T} \sum_{x \in \mathcal{B}} \mathbb{1}\{\text{argmax}\, p(x) = i\}$$

Fraction of tokens sent to expert i

$$P_i = \frac{1}{T} \sum_{x \in \mathcal{B}} p_i(x).$$

Fraction of router probability allocated to expert i

Expert Capacity

- We also introduce capacity of each expert, that should not be exceeded.
- Why? Tensor shapes must be fixed at compile time on TPUs/XLA backends, so we cannot have a dynamically-sized expert buffer. The capacity factor is a pre-allocated worst-case slot count

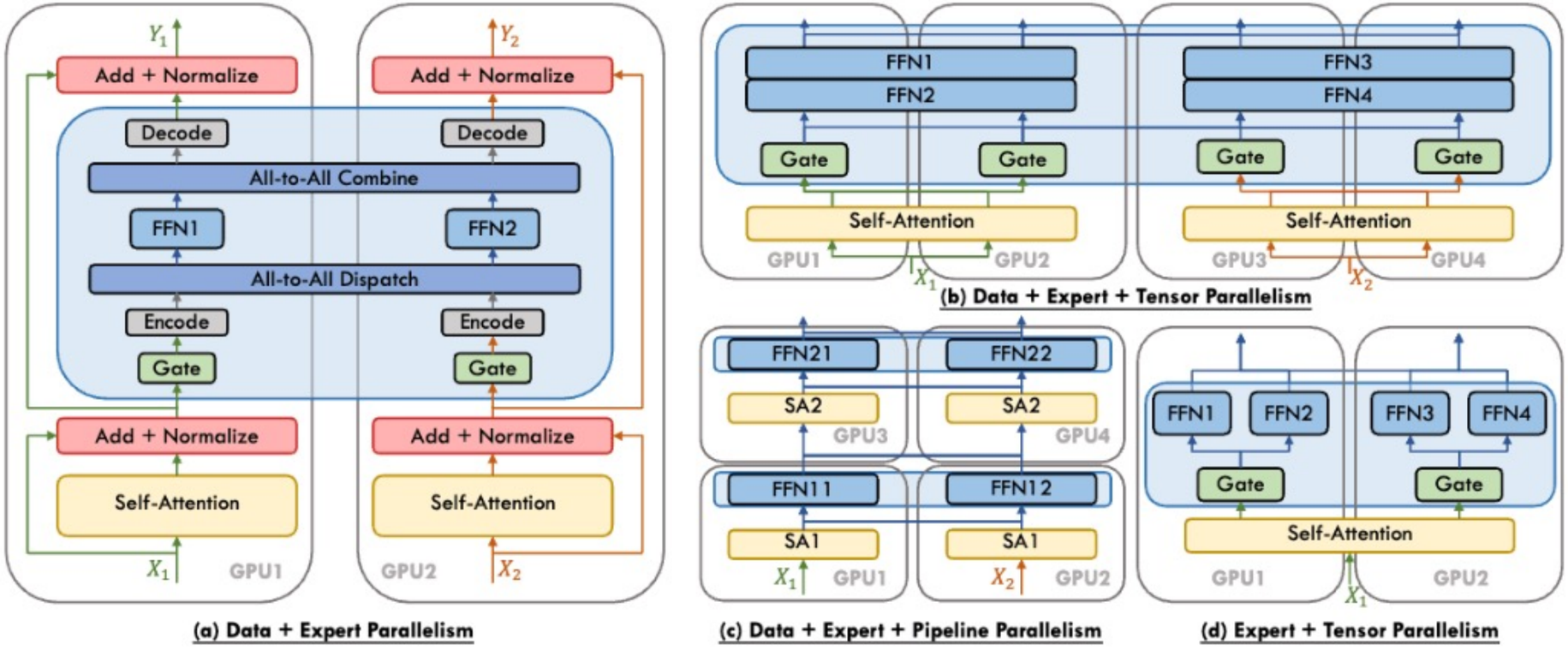
Expert Capacity

- We also introduce capacity of each expert, that should not be exceeded.
- Why? Tensor shapes must be fixed at compile time on TPUs/XLA backends, so we cannot have a dynamically-sized expert buffer. The capacity factor is a pre-allocated worst-case slot count
- If all experts have reached their capacity, the token will not be processed by any expert but instead sent to the next layer skipping the expert. This is referred to as token overflow.

$$\text{expert capacity} = \left(\frac{\text{tokens per batch}}{\text{number of experts}} \right) \times \text{capacity factor}.$$

Expert Parallelism

- **Idea:** since the FFN are fully independent we can simply put each expert's feedforward layer on a different worker



Active Parameters

- **We still need to load the entire model onto device but when we run inference, we only need to use a subset (active). MoE models need more VRAM to load in all experts but run faster during inference**

Active Parameters

- We still need to load the entire model onto device but when we run inference, we only need to use a subset (active). MoE models need more VRAM to load in all experts but run faster during inference

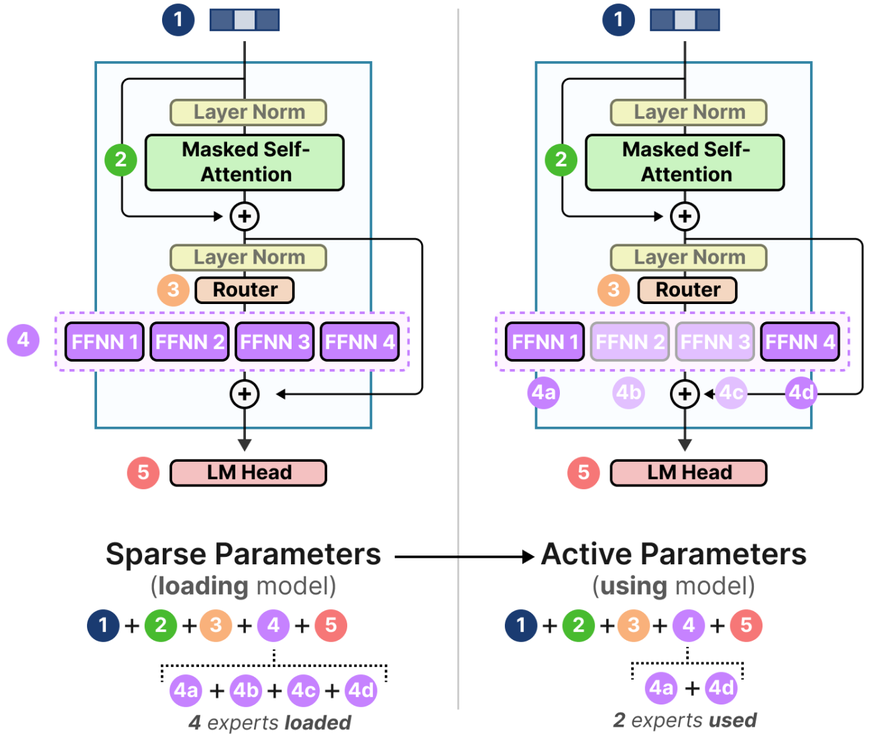
- For a Transformer with L layers, M of which are MoE layers, each with N experts of hidden size d_f :

$$\text{Total params} \approx L_{\text{dense}} \cdot 2d \cdot d_{\text{ff}} + M \cdot N \cdot 2d \cdot d_{\text{ff}}$$

$$\text{Active params per token} \approx L_{\text{dense}} \cdot 2d \cdot d_{\text{ff}} + M \cdot k \cdot 2d \cdot d_{\text{ff}}$$

$$\text{Ratio} = \frac{\text{Active}}{\text{Total}} \approx \frac{L_{\text{dense}} + M \cdot k}{L_{\text{dense}} + M \cdot N}$$

For Switch Transformer ($k = 1, N = 2048$, all layers MoE): ratio $\approx 1/2048$ - only $\mathbf{0.05\%}$ of parameters are active per token.



Recap

- Tensor Parallelism: split weight matrices, shard activations
- Pipeline Parallelism: split layers across nodes
- Context Parallelism: ring Attention for long sequences
- 5D Parallelism: TP + SP + PP + CP + DP
- MoE: N experts, Top-k gating, 5–8× capacity at same FLOPs. Load balancing is key

Next:

- Efficient Inference